C++ 프로그래밍 · 코딩 테스트

- 확인 및 시험

01 개발환경·변수·기본 문법

핵심 정의

변수: **변수**는 메모리의 주소에 붙이는 이름이며, **변수**를 만들면 타입에 따라 메모리 공간이 확보된다. [U01-017] (L01 Variable, 슬라이드 41)

초기화: **변수**를 정의하면서 초기값을 설정하는 것을 **초기화**라고 한다. [U01-018] (L01 Definition and Initialization, 슬라이드 42)

배열: **배열**은 동일한 타입인 요소들의 집합인 복합(compound) 데이터 타입이다. [U01-021] (L02 Array, 슬라이드 5)

반복문 제어

문장	동작
continue [U01-059] (L02 Looping Statement, 슬라이드 39)	continue는 이후 문장을 실행하지 않고 다음 iteration으로 곧바로 넘어간다. [U01-035] (L02 Looping Statement, 슬라이드 39)
break [U01-060] (L02 Looping Statement, 슬라이드 39)	break는 이후 문장을 실행하지 않고 루프 밖으로 바로 빠져나간다. [U01-036] (L02 Looping Statement, 슬라이드 39)

주요 수식·복잡도

byte와 bit: 1 byte = 8 bit = 2^8 = 256개의 숫자 표현 가능; 양수 범위 0~255 [U01-044] (L01 Bit, Byte, Hex, 슬라이드 35)

02 함수·포인터·참조·메모리

핵심 정의

포인터: **포인터**는 주소값을 저장하는 **변수**이다. [U02-016] (L04 Accessing Pointer Address, 슬라이드 10)

역참조: **역참조**는 * 연산자로 **포인터**의 주소에 저장된 데이터에 접근하는 것이다. [U02-019] (L04 Dereferencing a Pointer, 슬라이드 15)

참조자: **참조자**는 **변수**의 별명이며 선언과 동시에 **초기화**되어야 하고, 한 번 **초기화**되면 다른 **변수**의 **참조자**가 될 수 없다. [U02-021] (L04 Reference, 슬라이드 42)

인수 전달

방식	원문 사용 조건
Pass-by-value [U02-046] (L04 Pointers vs References, 슬라이드 51)	원본을 수정하지 않을 때, 크기가 작은 기본 자료형을 전달한다. [U02-047] (L04 Pointers vs References, 슬라이드 51)
Pass-by-address [U02-048] (L04 Pointers vs References, 슬라이드 52)	원본 수정이 필요하고, **포인터**가 **nullptr**이어도 상관없을 때 사용한다. [U02-049] (L04 Pointers vs References, 슬라이드 52)
Pass-by-reference [U02-050] (L04 Pointers vs References, 슬라이드 55)	원본을 수정하고, 매개**변수**가 null이 되지 않음이 보장될 때 사용한다. [U02-051] (L04 Pointers vs References, 슬라이드 55)

원문 문법 발췌

참조자 매개변수 - 원문 발췌 (원문 발췌)

```
int change_val(int &p) {
  p = 3;

  return 0;
}
```

[U02-042] (B 3장·참조자를 함수 인자로 받기, 인쇄 p.35)

03 클래스·생성·복사·const

핵심 정의

생성자: **생성자**는 **객체**가 생성될 때 자동 호출되는 특수한 멤버함수로, **클래스**와 이름이 같고 반환형이 없다. [U03-013] (L05A Constructors, 슬라이드 41)

소멸자: **소멸자**는 **객체**가 소멸할 때 자동 호출되며 **클래스** 이름 앞에 ~를 붙이고 반환형·매개**변수**가 없다. [U03-014] (L05A Destructor, 슬라이드 44)

explicit: **생성자**를 **explicit**으로 선언하면 그 **생성자**를 이용한 암시적 변환을 막을 수 있다. [U03-038] (B 5장·클래스의 explicit 과 mutable 키워드·explicit, 인쇄 p.183)

mutable: **mutable**로 선언한 멤버**변수**는 const 멤버함수 안에서도 값을 바꿀 수 있다. [U03-039] (B 5장·클래스의 explicit 과 mutable 키워드·mutable, 인쇄 p.183)

복사 방식

방식	복사 내용
얕은 복사 [U03-048] (L05B Shallow vs Deep Copy, 슬라이드 14)	얕은 복사는 **포인터**가 가리키는 데이터가 아니라 **포인터** 주소값 자체를 복사한다. [U03-025] (L05B Shallow vs Deep Copy, 슬라이드 14)
깊은 복사 [U03-049] (L05B Shallow vs Deep Copy, 슬라이드 20)	**깊은 복사**는 새로운 힙 공간을 할당한 뒤 동일한 데이터를 복사하여 복사 생성하는 방식이다. [U03-026] (L05B Shallow vs Deep Copy, 슬라이드 20)

04 상속·다형성

핵심 정의

상속: **상속**은 **기본 클래스**를 기반으로 새 **클래스**를 만드는 기능이다. [U04-003] (L06 (Summary) Inheritance, 슬라이드 53)

다형성: **다형성**(polymorphism)은 하나의 메소드를 호출했음에도 여러 가지 다른 작업을 하는 것이다. [U04-014] (B 7장·가상 함수와 다형성, 인쇄 p.321)

추상 클래스: **추상 클래스**(Abstract Class)는 **객체**를 생성할 수 없으며 **상속** 계층에서 **기본 클래스**로 사용되는 **클래스**이다. [U04-021] (L07 Pure Virtual Functions and Abstract Classes, 슬라이드 33)

바인딩

구분	시점	기준
정적 바인딩 [U04-049] (L07 Polymorphism, 슬라이드 13)	**컴파일** 시 [U04-050] (L07 Polymorphism, 슬라이드 13)	타입을 기준으로 호출 함수를 결정한다. [U04-051] (L07 Polymorphism, 슬라이드 13)
동적 바인딩 [U04-052] (L07 Polymorphism, 슬라이드 13)	런타임 [U04-053] (L07 Polymorphism, 슬라이드 13)	실제 메모리에 저장된 타입을 기준으로 결정한다. [U04-054] (L07 Polymorphism, 슬라이드 13)

05 연산자 오버로딩·N차원 배열 프로젝트

핵심 정의

연산자 오버로딩: **연산자 오버로딩**은 +, =, * 등의 연산자를 사용자 정의 타입에 대해 사용할 수 있게 하는 기능이다. [U05-003] (L08 Operator Overloading, 슬라이드 6)

Wrapper 클래스: **Wrapper 클래스**는 기본 자료형을 **클래스**로 포장하여 각 자료형을 **객체**로 사용하는 **클래스**이다. [U05-022] (B 6장·int Wrapper 클래스 - 타입 변환 연산자, 인쇄 p.233)

반복자: **반복자**(iterator)는 **배열**의 모든 원소를 순차적으로 접근할 수 있게 하며, ++로 다음 원소를 가리킨다. [U05-034] (B 6장·N 차원 배열 만들기 프로젝트·반복자, 인쇄 p.260)

전위·후위 증가

구분 형식	반환 내용
전위: operator++() [U05-055] (B 6장·전위/후위 증감 연산자, 인쇄 p.236)	**전위 증감** 연산자는 값을 바꾼 자기 자신을 반환한다. [U05-025] (B 6장·전위/후위 증감 연산자, 인쇄 p.236)
후위: operator++(int) [U05-056] (B 6장·전위/후위 증감 연산자, 인쇄 p.236)	**후위 증감** 연산자는 값이 바뀌기 이전의 **객체**를 반환한다. [U05-026] (B 6장·전위/후위 증감 연산자, 인쇄 p.236)

06 표준 입출력·파일·엑셀 프로젝트

핵심 정의

스트림 버퍼: **스트림 버퍼**는 데이터를 내보내거나 받아들이기 전에 임시로 저장하는 곳이다. [U06-004] (B C++ 입출력 라이브러리, 인쇄 p.341)

조작자: **조작자**는 스트림을 조작하여 입력 혹은 출력 방식을 바꾸는 함수이다. [U06-011] (B 형식 플래그 (format flag) 와 조작자(Manipulator), 인쇄 p.351)

후위 표기법: 중위 표기법은 피연산자 사이에 연산자를 넣으며, **후위 표기법**은 연산자를 피연산자 뒤에 두고 괄호 없이 표현한다. [U06-030] (B 엑셀 만들기 프로젝트 2부 / 중위 표기법과 후위 표기법, 인쇄 p.398)

스트림 오류 상태

플래그	내용
badbit [U06-046] (B istream 클래스, 인쇄 p.345)	스트림에 복구 불가능한 오류 발생 시 [U06-047] (B istream 클래스, 인쇄 p.345)
failbit [U06-048] (B istream 클래스, 인쇄 p.345)	스트림에 복구 가능한 오류 발생 시 [U06-049] (B istream 클래스, 인쇄 p.345)
eofbit [U06-050] (B istream 클래스, 인쇄 p.345)	입력 작업 시에 EOF 도달 시 [U06-051] (B istream 클래스, 인쇄 p.345)

원문 문법 발췌

파일 전체 읽기: 원문 발췌 (원문 발췌)

```
in.seekg(0, std::ios::end);
int size = in.tellg();
s.resize(size);
in.seekg(0, std::ios::beg);
in.read(&s[0], size);
```

[U06-042] (B 파일 전체를 한 번에 읽기, 인쇄 p.361)

07 템플릿·가변 인자·메타프로그래밍

핵심 정의

인스턴스화: **클래스** 템플릿에 인자를 전달하여 실제 코드를 생성하는 것을 **클래스 템플릿 인스턴스화**(class template instantiation)라고 한다. [U07-007] (B C++ 템플릿(template), 인쇄 p.418)

템플릿 특수화: **템플릿 특수화**(template specialization)는 일부 경우를 따로 처리하는 것이며, 전달할 템플릿 인자가 없어도 template<>를 남겨야 한다. [U07-009] (B 템플릿 특수화(template specialization), 인쇄 p.419)

TMP: 템플릿 메타프로그래밍(Template Meta Programming, **TMP**)은 타입을 가지고 **컴파일**타임에 생성되는 코드로 프로그래밍하는 것이다. [U07-024] (B 템플릿 메타프로그래밍(Template Meta Programming - TMP), 인쇄 p.463-464)

Generic Programming 구현

방법	원문의 설명
매크로(#define) [U07-040] (L09 Generic Programming using Macro, 슬라이드 6)	코드의 단순 대체 [U07-041] (L09 Generic Programming using Macro, 슬라이드 6)
템플릿 [U07-042] (L09 Generic Programming using Template, 슬라이드 15)	**컴파일러**가 설계도를 기반으로 함수·**클래스**를 생성하며, 사용하지 않는 경우 생성하지 않는다. [U07-043] (L09 Generic Programming using Template, 슬라이드 15)

원문 문법 발췌

Fold 합산: 원문 발췌 (원문 발췌)

```
template <typename... Ints>
int sum_all(Ints... nums) {
  return (... + nums);
}
```

[U07-045] (B Fold Expression, 인쇄 p.455)

08 STL 컨테이너·알고리즘·문자열

핵심 정의

반복자: **반복자**(iterator)는 컨테이너 원소에 접근할 수 있는 **포인터**와 같은 **객체**이다. [U08-008] (B 반복자(iterator), 인쇄 p.500)

서술자: **알고리즘**은 보통 시작 **반복자**와 끝 바로 뒤 **반복자**를 받으며, **서술자**(Predicate)에는 보통 bool을 반환하는 함수 **객체**를 전달한다. [U08-028] (B C++ 표준 알고리즘 라이브러리, 인쇄 p.556)

string_view: **string_view**는 문자열을 소유하지 않고 참조해서 읽으며, 가리키는 문자열이 소멸한 뒤 사용하면 Undefined behavior가 발생한다. [U08-042] (B string_view / 소유하지 않고 읽기만 한다!, 인쇄 p.609)

시퀀스 컨테이너

종류	원문 특징
vector [U08-051] (B 벡터(std::vector), 인쇄 p.500)	연속 저장; 임의 접근 O(1); 임의 위치 추가·제거 O(n) [U08-052] (B 벡터(std::vector), 인쇄 p.495,500)
list [U08-053] (B 리스트(list), 인쇄 p.514)	양방향 연결; []·at 없음; 위치를 찾은 뒤 추가·제거 O(1) [U08-054] (B 리스트(list), 인쇄 p.513-514)
deque [U08-055] (B 덱(deque), 인쇄 p.517)	양 끝 추가·제거 O(1); 임의 접근 O(1); 임의 위치 추가·제거 O(n) [U08-056] (B 덱(deque), 인쇄 p.517)

주요 수식·복잡도

partial_sort 복잡도: O(N log M); N: 전체 원소 수, M: 정렬하려는 부분의 크기. [U08-050] (B 정렬(sort, stable_sort, partial_sort), 인쇄 p.561)

09 예외 처리

핵심 정의

예외: **예외**(exception)는 정상적인 상황에서 벗어난 **예외**적인 상황이다. [U09-003] (B 예외란?, 인쇄 p.613)

스택 풀기: **스택 풀기**(stack unwinding)는 **catch**로 이동하면서 **스택**상에 정의된 **객체**들을 소멸시키는 과정이다. [U09-014] (B 스택 풀기(stack unwinding), 인쇄 p.621)

noexcept: **noexcept**는 함수가 **예외**를 발생시키지 않음을 명시하는 키워드이다. [U09-022] (B 예외를 발생시키지 않는 함수 - noexcept, 인쇄 p.627)

예외 처리 구문

구문	역할
throw [U09-032] (B 예외 발생시키기 - throw, 인쇄 p.616)	**예외 객체**를 던지고 처리 부분으로 이동한다. [U09-033] (B 예외 발생시키기 - throw, 인쇄 p.616)
try [U09-034] (B 예외 처리하기 - try와 catch, 인쇄 p.618)	**try** 안에는 **예외**가 발생할 만한 코드를 넣으며, **예외**가 없으면 **catch**는 무시한다. [U09-011] (B 예외 처리하기 - try와 catch, 인쇄 p.618)
catch [U09-035] (B 예외 처리하기 - try와 catch, 인쇄 p.618)	**catch**는 던진 **예외**를 받는 부분이며, 괄호 안에 정의된 **예외**의 꼴에 맞는 **객체**를 받는다. [U09-012] (B 예외 처리하기 - try와 catch, 인쇄 p.618)

원문 문법 발췌

out_of_range 처리: 원문 발췌 (원문 발췌)

```
try {
  data = vec.at(index);
} catch (std::out_of_range& e) {
  std::cout << "예외발생! " << e.what() << std::endl;
}
```

[U09-031] (B 예외 처리하기 - try와 catch, 인쇄 p.617)

10 우측값·이동·완벽한 전달

핵심 정의

우측값 레퍼런스: **우측값 레퍼런스**는 &&로 정의하며, 원문의 MyString(MyString&& str)은 우측값을 인자로 받는다. [U10-009] (B 우측값 레퍼런스, 인쇄 p.641)

std::move: **std::move**는 <utility>에서 제공되며, 이동 자체를 수행하지 않고 받은 **객체**를 우측값으로 변환한다. [U10-016] (B move 함수(move semantics), 인쇄 p.655-657)

완벽한 전달: **완벽한 전달**(perfect forwarding)은 wrapper가 받은 인자를 원래 호출과 같은 형태로 다른 함수에 전달하는 문제를 다룬다. [U10-021] (B 완벽한 전달(perfect forwarding), 인쇄 p.661-663,670)

레퍼런스 겹침

형태	결과
int& & [U10-037] (B 보편적 레퍼런스 / 레퍼런스 겹침 규칙, 인쇄 p.669)	int& [U10-038] (B 보편적 레퍼런스 / 레퍼런스 겹침 규칙, 인쇄 p.669)
int& && [U10-039] (B 보편적 레퍼런스 / 레퍼런스 겹침 규칙, 인쇄 p.669)	int& [U10-038] (B 보편적 레퍼런스 / 레퍼런스 겹침 규칙, 인쇄 p.669)
int&& & [U10-040] (B 보편적 레퍼런스 / 레퍼런스 겹침 규칙, 인쇄 p.669)	int& [U10-038] (B 보편적 레퍼런스 / 레퍼런스 겹침 규칙, 인쇄 p.669)
int&& && [U10-041] (B 보편적 레퍼런스 / 레퍼런스 겹침 규칙, 인쇄 p.669)	int&& [U10-042] (B 보편적 레퍼런스 / 레퍼런스 겹침 규칙, 인쇄 p.669)

원문 문법 발췌

move를 사용하는 my_swap: 원문 발췌 (원문 발췌)

```
template <typename T>
void my_swap(T &a, T &b) {
  T tmp(std::move(a));
  a = std::move(b);
  b = std::move(tmp);
}
```

[U10-036] (B move 함수(move semantics), 인쇄 p.659)

11 스마트 포인터·함수 객체

핵심 정의

RAII: **RAII**(Resource Acquisition Is Initialization)는 **자원** 관리를 **스택**에 할당한 **객체**를 통해 수행하는 방식이다. [U11-006] (B Resource Acquisition Is Initialization - RAII, 인쇄 p.674)

Callable: **Callable**은 ()를 붙여 호출할 수 있는 모든 것을 뜻하며, 함수·함수 **객체**·람다가 예로 나온다. [U11-027] (B Callable, 인쇄 p.705-706)

std::function: **std::function**은 **Callable**을 **객체** 형태로 보관하며, 템플릿 인자로 반환형과 함수 인자를 나타내는 함수 타입을 받는다. [U11-028] (B std::function, 인쇄 p.706-707)

스마트 포인터

종류	소유·참조
unique_ptr [U11-041] (B 객체의 유일한 소유권 - unique_ptr, 인쇄 p.675)	특정 **객체**의 유일한 소유권 [U11-042] (B 객체의 유일한 소유권 - unique_ptr, 인쇄 p.675)
shared_ptr [U11-043] (B shared_ptr, 인쇄 p.688)	여러 **포인터**가 같은 **객체**를 소유 [U11-044] (B shared_ptr, 인쇄 p.688)
weak_ptr [U11-045] (B weak_ptr, 인쇄 p.700)	참조 개수를 늘리지 않으며 **shared_ptr**로 변환하여 접근 [U11-046] (B weak_ptr, 인쇄 p.700)

12 쓰레드·동기화·atomic·비동기·ThreadPool

핵심 정의

쓰레드: 같은 **프로세스** 안의 쓰레드들은 메모리를 공유한다. [U12-004] (B 16장 / 멀티 쓰레드 프로그램, 인쇄 p.722)

lock_guard: **lock_guard** 는 **생성자**에서 뮤텍스를 lock 하고 소멸할 때 unlock 한다. [U12-012] (B 16장 / lock_guard, 인쇄 p.748)

future::get: future 의 get 은 안의 데이터를 이동하므로 두 번 호출하면 안 된다. [U12-028] (B 16장 / std::promise 와 std::future, 인쇄 p.795)

비동기 도구

도구	원문 기능
shared_future [U12-049] (B 16장 / shared_future, 인쇄 p.799)	여러 쓰레드에서 future 를 get 해야 하는 경우 shared_future 를 사용한다. [U12-031] (B 16장 / shared_future, 인쇄 p.799)
packaged_task [U12-050] (B 16장 / packaged_task, 인쇄 p.801)	**packaged_task** 는 전달받은 함수를 실행하여 리턴값을 promise 에 설정하고, get_future 로 대응하는 future 를 얻는다. [U12-032] (B 16장 / packaged_task, 인쇄 p.801)
std::async [U12-051] (B 16장 / std::async, 인쇄 p.802)	std::async 는 실행 결과를 future 로 돌려주며, launch::async 는 바로 새 쓰레드에서 실행하고 launch::deferred 는 future::get 시 새 쓰레드 없이 실행한다. [U12-033] (B 16장 / std::async, 인쇄 p.803)

13 유니폼 초기화·constexpr·decltype

핵심 정의

유니폼 초기화: **유니폼 초기화**는 {} 를 이용한 **생성자** 호출이며, 인자 없는 **생성자**가 함수의 정의로 오해되는 일을 막는다. [U13-001] (B 17장 / 뭘 배웠지?, 인쇄 p.836)

constexpr: **constexpr** 은 식이 **상수**식이라고 명시하는 키워드이며, **객체** 정의에 붙이면 그 **객체**를 **상수**식에 사용할 수 있다. [U13-012] (B 18장 / constexpr, 인쇄 p.838)

std::declval: **std::declval**<T>() 는 T 의 **생성자**를 직접 호출하지 않고 타입 연산에 **객체**를 나타낼 수 있으며, 런타임에서 사용하면 오류가 난다. [U13-031] (B 19장 / std::declval, 인쇄 p.863)

식별자 표현식이 아닌 식의 decltype

값 카테고리	decltype 결과 (식의 타입 T)
xvalue [U13-033] (B 19장 / decltype, 인쇄 p.853)	T&& [U13-034] (B 19장 / decltype, 인쇄 p.853)
lvalue [U13-035] (B 19장 / decltype, 인쇄 p.853)	T& [U13-036] (B 19장 / decltype, 인쇄 p.853)
prvalue [U13-037] (B 19장 / decltype, 인쇄 p.853)	T [U13-038] (B 19장 / decltype, 인쇄 p.853)

14 type_traits·정규표현식·난수·시간·파일시스템·유틸리티

핵심 정의

SFINAE: **SFINAE** 는 Substitution Failure Is Not An Error 이며, 문법에 맞지 않는 템플릿 인자 치환 결과를 **함수 오버로딩** 후보에서 제외한다. [U14-007] (B 20장 / SFINAE, 인쇄 p.878)

optional: **std::optional** 은 원하는 값을 보관할 수도, 보관하지 않을 수도 있는 **클래스**이다. [U14-031] (B 20장 / std::optional, 인쇄 p.940)

variant: **std::variant** 는 **컴파일** 타임에 정한 여러 타입 중 한 타입의 **객체**를 보관하는 one-of **클래스**이다. [U14-035] (B 20장 / std::variant, 인쇄 p.946)

값의 묶음

클래스	원문 기능
optional [U14-047] (B 20장 / std::optional, 인쇄 p.940)	**std::optional** 은 원하는 값을 보관할 수도, 보관하지 않을 수도 있는 **클래스**이다. [U14-031] (B 20장 / std::optional, 인쇄 p.940)
variant [U14-048] (B 20장 / std::variant, 인쇄 p.946)	**std::variant** 는 **컴파일** 타임에 정한 여러 타입 중 한 타입의 **객체**를 보관하는 one-of **클래스**이다. [U14-035] (B 20장 / std::variant, 인쇄 p.946)
tuple [U14-049] (B 20장 / std::tuple, 인쇄 p.950)	**std::tuple** 은 서로 다른 타입의 **객체**들을 묶으며, make_tuple 로 만들고 std::get 으로 원소에 접근한다. [U14-037] (B 20장 / std::tuple, 인쇄 p.950-951)

15 Visual Studio 디버깅·Make

핵심 정의

디버깅: **디버깅**은 버그(오류)를 제거하는 것이다. [U15-003] (L02D What is debugging?, 슬라이드 3)

make: **make** 는 주어진 쉘 명령어를 조건에 맞게 실행하며, 조건을 담은 파일을 Makefile 이라고 한다. [U15-002] (B 22장 / make, 인쇄 p.964)

prerequisites: **prerequisites** 는 타겟에 사용할 파일의 목록이며 의존 파일(dependency)이라고도 한다. [U15-019] (B 22장 / 필요조건들(prerequisites), 인쇄 p.964)

강의의 디버깅 진행 키

키	동작
F10 [U15-037] (L02D Break point, 슬라이드 6)	step over: 해당 **명령문** 실행 후 다음 명령줄로 진행 [U15-038] (L02D Break point, 슬라이드 6)
F11 [U15-039] (L02D Break point, 슬라이드 6)	step into: **명령문**에서 호출하는 추가 코드로 들어감 [U15-040] (L02D Break point, 슬라이드 6)
Shift+F11 [U15-041] (L02D Break point, 슬라이드 6)	step out: 상위 stack 으로 나가기 [U15-042] (L02D Break point, 슬라이드 6)

16 리스트, 스택, 큐

핵심 정의

시간 복잡도: **시간 복잡도**는 특정 작업을 수행하는 데 걸리는 시간을 데이터 크기에 대한 수식으로 표현하는 방식이다. [U16-003] (K 1.2 연속된 자료 구조와 연결된 자료 구조, PDF p.12)

반복자: **반복자**는 **포인터**와 비슷하지만 **STL** 컨테이너에 대해 공통의 인터페이스를 제공한다. [U16-021] (K 1.6 반복자, PDF p.39)

컨테이너 어댑터: **컨테이너 어댑터**는 기존 컨테이너를 감싸 해당 자료 구조에 필요한 인터페이스만 제공한다. [U16-027] (K 1.9 컨테이너 어댑터, PDF p.57)

주요 컨테이너 처리 순서

자료 구조	처리
스택 [U16-050] (K 1.9, PDF p.56)	LIFO, 한쪽 끝에서 삽입·삭제 [U16-051] (K 1.9, PDF p.56)
큐 [U16-052] (K 1.9.2, PDF p.58)	FIFO, 맨 뒤에 추가·맨 앞에서 제거 [U16-053] (K 1.9.2, PDF p.58)

주요 수식·복잡도

연속된 자료 구조의 i번째 원소 주소: BA + i * sizeof(type); BA: 시작 주소, sizeof(type): 원소 하나에 필요한 메모리 크기 [U16-040] (K 1.2.1 연속된 자료 구조, PDF p.13)

17 트리, 힙, 그래프

핵심 정의

BST: **BST**는 왼쪽 노드 ≤ 부모 노드 ≤ 오른쪽 노드의 관계를 가진다. [U17-010] (K 2.4.1 이진 검색 트리, PDF p.74)

높이: **BST**의 레벨 수를 **높이**(height)라고 하며 원소 검색의 실제 **시간 복잡도**는 O(**높이**)이다. [U17-012] (K 2.4.4 균형 트리, PDF p.85)

최대 힙: **최대 힙**은 최대 원소를 루트에 두고 부모 노드가 두 자식 노드보다 항상 커야 한다는 불변성을 유지한다. [U17-018] (K 2.5 힙, PDF p.89)

트리 순회

방법	순서
전위 순회 [U17-041] (K 2.3.2, PDF p.70)	현재 → 왼쪽 → 오른쪽 [U17-042] (K 2.3.2, PDF p.70)
중위 순회 [U17-043] (K 2.3.2, PDF p.71)	왼쪽 → 현재 → 오른쪽 [U17-044] (K 2.3.2, PDF p.71)
후위 순회 [U17-045] (K 2.3.2, PDF p.71)	왼쪽 → 오른쪽 → 현재 [U17-046] (K 2.3.2, PDF p.71)

주요 수식·복잡도

배열에 저장한 힙의 인덱스: 자식: 2*i + 1, 2*i + 2 / 부모: (i - 1)/2; 부모 또는 자식이 i번째 인덱스일 때 [U17-040] (K 2.5 힙, PDF p.89)

18 해시 테이블과 블룸 필터

핵심 정의

룩업: **룩업**(**lookup**, 조회)은 특정 원소의 존재를 확인하거나 특정 키에 해당하는 값을 찾는 작업이다. [U18-003] (K 3.1 들어가며, PDF p.107)

충돌: **충돌**은 서로 다른 키에 대해 해시 함수가 같은 해시 값을 반환하는 현상이다. [U18-007] (K 3.2.1 해싱, PDF p.109)

블룸 필터: **블룸 필터**는 공간 효율이 높지만 거짓-긍정(false positive)이 나올 수 있으며 거짓-부정(false negative)은 없다. [U18-024] (K 3.5 블룸 필터, PDF p.138)

블룸 필터 룩업

반환	원문 의미
true [U18-046] (K 3.5, PDF p.138)	있을 수도 있고 없을 수도 있음 [U18-047] (K 3.5, PDF p.138)
false [U18-048] (K 3.5, PDF p.138)	확실히 없음 [U18-049] (K 3.5, PDF p.138)

주요 수식·복잡도

부하율: **부하율** = 전체 키 개수 / 해시 테이블 크기; 각 리스트에 저장되는 키의 평균 개수 [U18-036] (K 3.3.2 연습 문제 14, PDF p.118)

19 분할 정복

핵심 정의

분할 정복: **분할 정복**은 문제를 작은 부분 문제로 나누고 각각의 해답을 구한 뒤 합쳐 전체 해답을 얻는다. [U19-004] (K 4.1 들어가며, PDF p.149)

맵: 맵은 컨테이너 C의 모든 원소에 함수 f(x)를 적용하는 연산이다. [U19-025] (K 4.6.1 맵과 리듀스 추상화, PDF p.180)

리듀스: **리듀스**는 모든 원소 x에 함수 f(acc,x)를 적용하여 하나의 값으로 축약하는 연산이다. [U19-026] (K 4.6.1 맵과 리듀스 추상화, PDF p.180)

그림 4-9 병합 정렬과 퀵 정렬의 점근적 시간 복잡도

알고리즘	최선의 경우	평균	최악의 경우
병합 정렬 [U19-038] (K 4.4.4 그림 4-9, PDF p.168)	O(nlogn) [U19-039] (K 4.4.4 그림 4-9, PDF p.168)	O(nlogn) [U19-039] (K 4.4.4 그림 4-9, PDF p.168)	O(nlogn) [U19-039] (K 4.4.4 그림 4-9, PDF p.168)
퀵 정렬 [U19-040] (K 4.4.4 그림 4-9, PDF p.168)	O(nlogn) [U19-039] (K 4.4.4 그림 4-9, PDF p.168)	O(nlogn) [U19-039] (K 4.4.4 그림 4-9, PDF p.168)	$O(n^2)$ [U19-041] (K 4.4.4 그림 4-9, PDF p.168)

주요 수식·복잡도

선형 시간 선택의 복잡도: O(n); 원문의 다섯 원소 묶음과 중앙값 피벗을 사용하는 **알고리즘** [U19-037] (K 4.4.6~4.4.7 선형 시간 선택, PDF p.177)

20 그리디 알고리즘

핵심 정의

최적 부분 구조: **최적 부분 구조**는 문제의 최적 솔루션이 부분 문제들의 최적 솔루션으로 구성되는 속성이다. [U20-011] (K 5.3.5 그리디 알고리즘의 요구 조건, PDF p.207)

그리디 선택: **그리디 선택**은 지역적 최적 솔루션을 반복 선택하여 전체 최적 솔루션을 구할 수 있는 속성이다. [U20-012] (K 5.3.5 그리디 알고리즘의 요구 조건, PDF p.207)

최소 신장 트리: **최소 신장 트리**는 **그래프**의 모든 정점을 연결하며 에지 가중치 합이 최소인 트리이다. [U20-014] (K 5.3.6 최소 신장 트리 문제, PDF p.208)

그리디 선택 기준

문제	기준
최단 작업 우선 [U20-046] (K 5.2.1, PDF p.197)	처리 시간이 짧은 순 [U20-047] (K 5.2.1, PDF p.197)
분할 가능 배낭 [U20-041] (K 5.3.2, PDF p.201)	단위 무게당 가격이 높은 순 [U20-048] (K 5.3.2, PDF p.201)
크루스칼 MST [U20-049] (K 5.3.6, PDF p.209)	사이클을 만들지 않는 최소 가중치 에지 [U20-050] (K 5.3.6, PDF p.209)

21 그래프 알고리즘 I

핵심 정의

BFS: 너비 우선 탐색(**BFS**, Breadth-First Search)은 시작 정점을 경계(frontier)에 추가한 뒤, 현재 경계에 인접한 정점을 반복적으로 탐색한다. [U21-008] (K 6.2.1, PDF p.236)

DFS: 깊이 우선 탐색(**DFS**, Depth-First Search)은 시작 정점에서 특정 경로를 따라 가능한 멀리 있는 정점을 재귀적으로 먼저 방문한다. [U21-011] (K 6.2.3, PDF p.243)

MST: **최소 신장 트리**(MST) 문제는 주어진 **그래프**의 모든 정점을 연결하고, 연결된 에지의 가중치 합이 최소인 트리 T를 구하는 문제이다. [U21-018] (K 6.3, PDF p.253)

프림과 다익스트라

알고리즘	거리 값·종료 조건
프림 [U21-053] (K 6.4, PDF p.260)	경계로부터 최소 거리·모든 정점 방문 후 종료 [U21-054] (K 6.4, PDF p.260)
다익스트라 (음수 가중치 없음) [U21-055] (K 6.4, PDF p.260)	시작 정점으로부터 전체 거리·목적 정점이 나타나면 종료 [U21-056] (K 6.4, PDF p.260)

주요 수식·복잡도

BFS와 **DFS**의 **시간 복잡도**: O(V + E); V는 정점의 개수, E는 에지의 개수이다. [U21-038] (K 6.2.2·6.2.4, PDF p.243,249)

22 그래프 알고리즘 II

핵심 정의

음수 가중치 사이클: **음수 가중치 사이클**(negative weight cycle)은 에지 가중치의 합이 음수가 되는 사이클이다. [U22-008] (K 7.4, PDF p.277)

강한 연결 요소: 방향 **그래프**를 같은 구역의 정점끼리 서로 이동 가능한 경로를 갖도록 나눌 때, 각 구역이 **강한 연결 요소**를 나타낸다. [U22-020] (K 7.6, PDF p.297)

그래프의 전치: **그래프**의 **전치**(transpose)는 시작 정점과 목표 정점을 서로 뒤바꿔 각 에지의 방향을 반대로 하는 것이다. [U22-024] (K 7.7, PDF p.300)

7.8에서 제시한 최단 거리 알고리즘 선택

구하려는 값	본문의 선택
모든 정점 사이의 최단 거리 [U22-041] (K 7.8, PDF p.311)	**존슨 알고리즘** [U22-042] (K 7.8, PDF p.311)
하나의 시작 정점에서 나머지 정점까지의 최단 거리 [U22-043] (K 7.8, PDF p.311)	존슨보다 벨만-포드가 효율적 [U22-044] (K 7.8, PDF p.311)

주요 수식·복잡도

존슨 알고리즘의 에지 가중치 변환: W(uv) = w(uv) + d[s, u] - d[s, v]; w(uv)는 초기 에지 가중치, d[s, u]와 d[s, v]는 더미 정점 s로부터의 최단 거리, W(uv)는 변환된 가중치이다. [U22-039] (K 7.5, PDF p.286)

23 동적 계획법 I

핵심 정의

중복되는 부분 문제: **중복되는 부분 문제**는 같은 부분 문제가 반복적으로 나타나는 특성이다. [U23-003] (K 8.2 동적 계획법이란?, PDF p.318)

최적 부분 구조: **최적 부분 구조**는 전체 문제의 최적해를 부분 문제의 최적해 조합으로 표현할 수 있는 특성이다. [U23-004] (K 8.2 동적 계획법이란?, PDF p.318)

기저 조건: **기저 조건**(base case)은 더 이상의 재귀 없이 해를 구할 수 있는 지점이다. [U23-006] (K 8.2 동적 계획법이란?, PDF p.316)

동적 계획법의 두 접근

방법	원문 설명
메모이제이션 [U23-040] (K 8.3 메모이제이션: 하향식 접근 방법, PDF p.319)	**메모이제이션**(memoization)은 하향식 방식에서 부분 문제의 해를 캐시에 넣어 사용하는 기술이다. [U23-008] (K 8.3 메모이제이션: 하향식 접근 방법, PDF p.319)
타뷸레이션 [U23-041] (K 8.4 타뷸레이션: 상향식 접근 방법, PDF p.320)	**타뷸레이션**(tabulation)은 **기저 조건** 해답부터 시작하여 부분 문제의 해답을 표에 저장하고 재사용하는 방식이다. [U23-012] (K 8.4 타뷸레이션: 상향식 접근 방법, PDF p.320)

주요 수식·복잡도

피보나치 수열의 **기저 조건**과 재귀 관계: F(0) = 0
F(1) = 1
F(n) = F(n-1) + F(n-2); 세 번째 이후 원소는 바로 앞 두 원소의 합이다. [U23-038] (K 8.2 동적 계획법이란?, PDF p.316)

24 동적 계획법 II

핵심 정의

상태 공간 축소: **상태 공간 축소**(state space reduction)는 **상태**를 표현하는 공간의 크기를 최소화하도록 동적 계획법 **알고리즘**을 재구성하는 작업이다. [U24-013] (K 9.4.4 상태 공간 축소, PDF p.389)

단일 시작 최단 경로: **단일 시작 최단 경로 문제**(single-source shortest **path** problem)는 한 정점에서 나머지 모든 정점까지의 최단 경로를 찾는다. [U24-016] (K 9.5.2 동적 계획법으로 최단 경로 문제 다루기, PDF p.405)

모든 쌍 최단 경로: **모든 쌍 최단 경로** 문제(all-pairs shortest **path** problem)는 모든 정점 사이의 최단 경로를 계산한다. [U24-017] (K 9.5.2 동적 계획법으로 최단 경로 문제 다루기, PDF p.405)

배낭 문제

문제	상태 전환의 차이
0-1 배낭 [U24-036] (K 9.4.4 상태 공간 축소, PDF p.394)	i번째 물건 포함 여부를 결정하기 위해 (i - 1)개 물건의 **상태**를 고려한다. [U24-037] (K 9.4.4 상태 공간 축소, PDF p.394)
무한 개수 배낭 [U24-038] (K 9.4.4 상태 공간 축소, PDF p.394)	같은 물건을 중복 포함할 수 있고, **상태** 전환은 무게에만 의존하므로 1차원 DP로 표현할 수 있다. [U24-039] (K 9.4.4 상태 공간 축소, PDF p.394)

주요 수식·복잡도

플로이드-워셜 **알고리즘** 복잡도: 시간 $O(V^3)$ · 메모리 $O(V^2)$; V는 **그래프** 정점 수이다. [U24-035] (K 9.5.4 모든 쌍 최단 경로, PDF p.413)

25 실습 문제 풀이

핵심 정의

구간 합 배열: 여행 경로의 추가 점수 풀이에서는 **구간 합 배열**(prefix sum array)을 사용하여 안쪽 for 반복문으로 계산하던 구간 합을 **상수** 시간에 계산한다. [U25-031] (K A.8.1 여행 경로, PDF p.499)

URLService: **URLService**는 짧은 URL을 키로, 원본 URL을 데이터 값으로 사용하여 맵에 함께 저장한다. [U25-012] (K A.3.1 긴 URL을 짧은 URL로 매핑하기, PDF p.447)

최대 이익의 상태: 최대 이익의 상향식 **상태**는 현재 인덱스, 상품 구매 비용, 상품 개수로 정의되며, 구현은 0-1 **배낭 문제** 풀이에서 차원이 하나 추가된 형태이다. [U25-038] (K A.9.1 최대 이익, PDF p.519-520)

LCS 구현 인터페이스

함수	원문 설명
LCS_Memoization() [U25-052] (K A.8.2 메모이제이션을 이용하여 최장 공통 부분 시퀀스 찾기, PDF p.501)	int형 LCS 길이; 2차원 정수 벡터 참조형 memo에 결과를 저장한다. [U25-053] (K A.8.2 메모이제이션을 이용하여 최장 공통 부분 시퀀스 찾기, PDF p.501)
LCS_Tabulation() [U25-054] (K A.8.3 타뷸레이션을 이용하여 최장 공통 부분 시퀀스 찾기, PDF p.505)	string형 **최장 공통 부분 시퀀스** 문자열을 반환한다. [U25-055] (K A.8.3 타뷸레이션을 이용하여 최장 공통 부분 시퀀스 찾기, PDF p.505)

주요 수식·복잡도

구간 합 배열: sum[i + 1] = sum[i] + distance[i]
[L, R] 구간의 원소 합: sum[R+1] - sum[L]; 합을 구할 전체 데이터 개수보다 하나 큰 **배열** sum을 선언하고 모든 원소를 0으로 **초기화**한다. [U25-050] (K A.8.1 여행 경로, PDF p.499)

멜로디 음 부분집합 수: $2^{12} = 4096$; 각 음을 2의 승수로 간주하여 비트 단위 연산으로 부분집합을 표현한다. [U25-051] (K A.8.4 멜로디 순열, PDF p.512)